EL PARAÍSO

Aplicación web de venta de entradas, licores y reservas de mesa

Documentación del proyecto y ciclo de vida del software

Dato	Información
Empresa	El Paraíso (Paraíso Club), Montería, Colombia
Sitio publicado	https://el-paraiso-discoteca.onrender.com
Repositorio	https://github.com/CarlosSob19/el-paraiso-discoteca
Integrantes	[Nombre 1] — [Nombre 2]
Asignatura y docente	[Asignatura] — [Docente]
Institución y fecha	[Institución] — [Fecha]

1. Introducción

Este documento describe el proceso de creación de la aplicación web de El Paraíso: el análisis de la necesidad, los requerimientos, el diseño de la base de datos y de las interfaces, la implementación, las pruebas y el despliegue en un servidor en línea.

2. La empresa

El Paraíso, que también se presenta como **Paraíso Club**, es un negocio de rumba de Montería registrado en Facebook en la categoría Bar. Los datos siguientes provienen de su página pública.

Dato	Información pública
Nombre	El Paraíso | Montería (Paraíso Club)
Dirección	Calle 9 Mz 15 Lote 10, Montería, Colombia, 230004
Seguidores en Facebook	Alrededor de 10 mil
Actividades publicadas	Noches de champeta, sábados de rumba, fiesta del 31 de octubre y partidos en pantalla gigante (clásico español, Selección Colombia)

3. Análisis de la problemática

En la información pública de la empresa, los eventos se anuncian por publicaciones de Facebook. Allí no aparece un catálogo con precios, un canal de venta de entradas, una forma de reservar mesa ni un registro de clientes. A partir de esa observación se identifican estas dificultades en el manejo de la información:

- La programación queda dispersa entre publicaciones y no se consulta en un solo lugar.
- No hay un registro centralizado de ventas, cupos ni clientes.
- Las reservas de mesa no tienen un control visible de disponibilidad por fecha, lo que puede causar dobles reservas.
- La administración de eventos, licores y promociones no se apoya en un sistema de información.

Para completar: estas dificultades salen de lo que se ve en la página pública. Confírmalas con el administrador del negocio y agrega cómo trabajan hoy (cuaderno, WhatsApp, Excel), a quién entrevistaste y en qué fecha.

4. Objetivos

Objetivo general: desarrollar una aplicación web que centralice la información de eventos, ventas, licores, reservas y usuarios de El Paraíso.

- Publicar la programación con sus tipos de entrada y cupos.
- Permitir la compra de entradas y licores y la reserva de mesas por fecha.
- Dar al administrador un panel para gestionar la información.
- Proteger el acceso con autenticación y validaciones de seguridad.

5. Requerimientos

5.1 Funcionales

Código	Requerimiento	Módulo
RF1	Registrar usuarios, iniciar y cerrar sesión	/registro, /login, /logout
RF2	Crear, consultar, editar y eliminar usuarios (administrador)	/admin/usuarios
RF3	Administrar eventos y sus precios por tipo de entrada	/admin/eventos
RF4	Administrar el catálogo de licores	/admin/licores
RF5	Administrar promociones	/admin/promociones
RF6	Administrar la galería	/admin/galeria
RF7	Reservar mesa según disponibilidad por fecha	/reservas
RF8	Consultar, confirmar, cancelar y eliminar reservas	/admin/reservas
RF9	Carrito para entradas, licores y reservas, con compra	/carrito, /checkout
RF10	Consultar el historial de compras	/mis_ordenes

5.2 No funcionales

- **Seguridad:** contraseñas cifradas, protección CSRF, límite de intentos y control de acceso por rol.
- **Usabilidad:** interfaz adaptable a celular y mensajes claros de error y confirmación.
- **Disponibilidad:** ejecución en un servidor web en línea.
- **Mantenibilidad:** código organizado por secciones y plantillas reutilizables.

5.3 Cumplimiento de lo solicitado

Requisito del trabajo	Cómo se cumple
Autenticación y sesiones	Flask-Login, contraseñas con hash y cookies protegidas
Creación y administración de usuarios	Registro público y módulo /admin/usuarios
Conexión y gestión de base de datos	Flask-SQLAlchemy; 9 tablas relacionadas
Registrar, consultar, modificar y eliminar	Usuarios, eventos, licores, promociones, galería y reservas
Interfaz funcional y amigable	Plantillas Jinja2 adaptables y navegación por módulos
Colores de la identidad de la empresa	Ver la sección 9
Validaciones de seguridad	Ver la sección 11
Organización de la información	Modelo relacional normalizado por entidades
Cierre de sesión seguro	Cierre por POST con token, limpieza del carrito y páginas privadas sin caché
Documentación del ciclo de vida	Este documento
Ejecución en servidor en línea	Render (sección 13)

6. Ciclo de vida del software

El proyecto se organiza con un modelo **incremental e iterativo**: la aplicación se divide en módulos (eventos, licores, galería, promociones, reservas, usuarios y compra), y cada uno pasa por análisis, diseño, implementación y pruebas.

Fase	Actividades	Resultado
1. Análisis	Revisión de la información de la empresa e identificación de la necesidad	Problemática y requerimientos
2. Diseño	Modelo entidad-relación, arquitectura, interfaces y paleta de color	Modelo de datos y plantillas
3. Implementación	Desarrollo con Flask, SQLAlchemy, HTML, CSS y JavaScript; control de versiones con Git	Aplicación funcional
4. Pruebas	Pruebas automatizadas de flujos, seguridad y base de datos; revisión en navegador	84 verificaciones correctas
5. Despliegue	Publicación desde GitHub en Render con base de datos en la nube	Sitio en línea
6. Mantenimiento	Corrección de errores y ajuste de los datos reales de la empresa	Versiones actualizadas

Para completar: agrega las fechas reales de cada fase y quién hizo cada actividad.

7. Arquitectura

La aplicación sigue el patrón cliente-servidor con una organización de tipo modelo-vista-controlador:

- **Modelo:** clases de SQLAlchemy en app.py, que corresponden a las tablas.
- **Vista:** plantillas Jinja2 en la carpeta templates, con estilos y scripts en static.
- **Controlador:** rutas de Flask en app.py, que validan los datos y responden al navegador.

El usuario entra desde el navegador; el servidor Gunicorn ejecuta Flask en Render, y Flask consulta la base de datos mediante SQLAlchemy. Por eso la misma aplicación funciona con SQLite, MySQL o PostgreSQL cambiando solo la variable DATABASE_URL.

8. Modelo entidad-relación

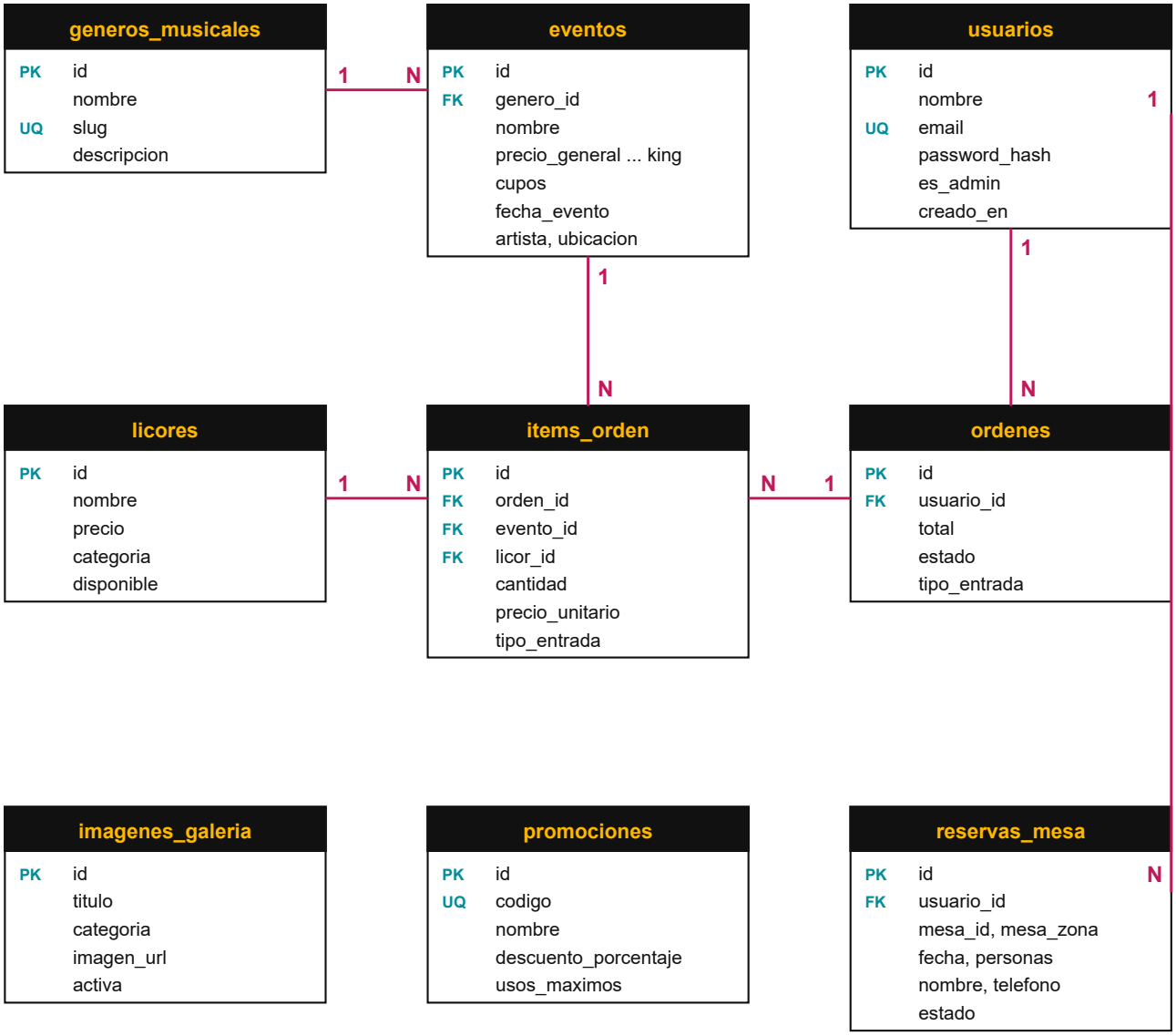


Figura 1. Modelo entidad-relación. PK: llave primaria, FK: llave foránea, UQ: valor único. La línea del margen derecho une usuarios con reservas_mesa.

Descripción de las tablas

Tabla	Propósito
usuarios	Cuentas de clientes y administradores
generos_musicales	Clasificación de los eventos
eventos	Programación, precios por tipo de entrada y cupos
licores	Catálogo de botellas
ordenes	Compra de un usuario; estado pendiente mientras es carrito
items_orden	Detalle de cada orden: entrada, licor o reserva
reservas_mesa	Reservas por mesa y fecha, con su estado
promociones	Códigos de descuento
imagenes_galeria	Fotos publicadas en la galería

Relaciones

- Un género tiene muchos eventos.
- Un usuario tiene muchas órdenes y puede tener muchas reservas.
- Una orden tiene muchos ítems.
- Un evento y un licor pueden aparecer en muchos ítems.

Los scripts **elparaiso.sql** (MySQL) y **elparaiso_postgres.sql** (PostgreSQL) crean las 9 tablas y cargan los datos iniciales.

9. Diseño de interfaces y teoría del color

Las capturas se tomaron en un entorno de prueba, así que usuarios, licores y reservas son datos de demostración.

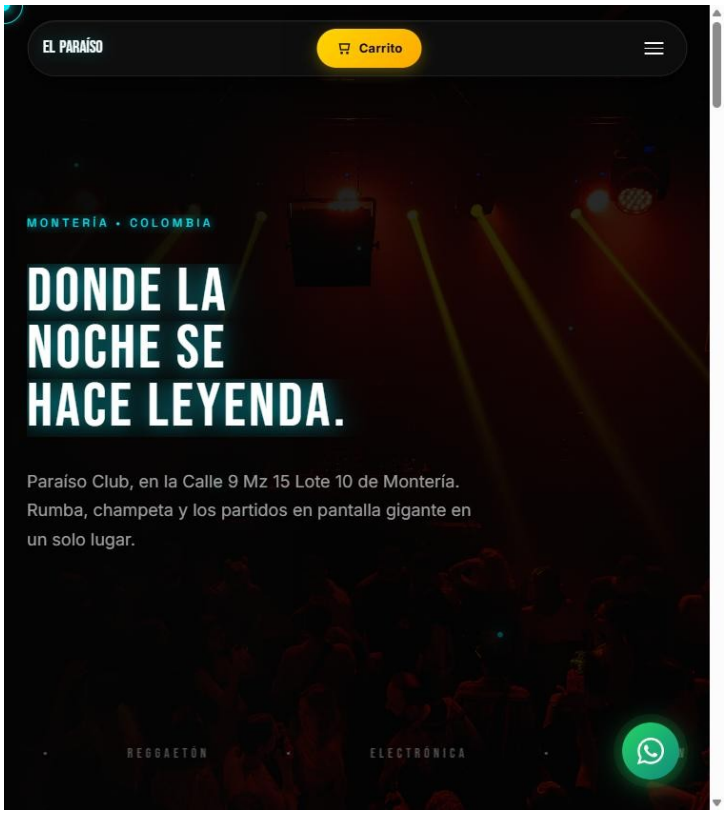


Figura 2. Página de inicio, vista de celular

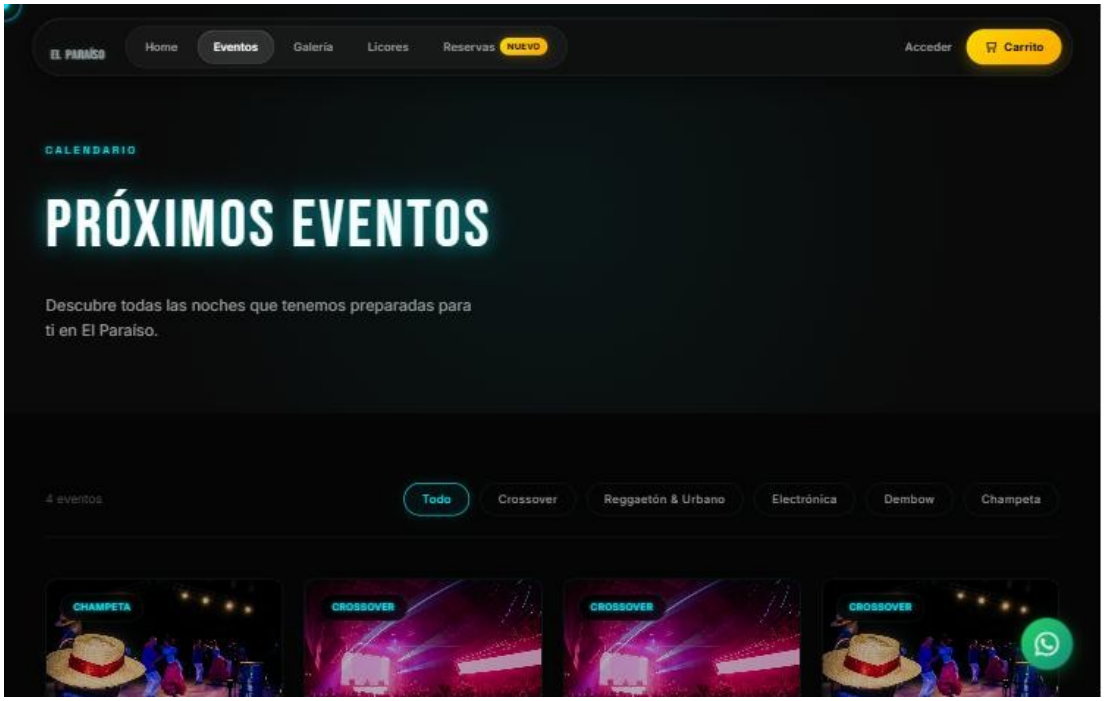


Figura 3. Calendario de eventos

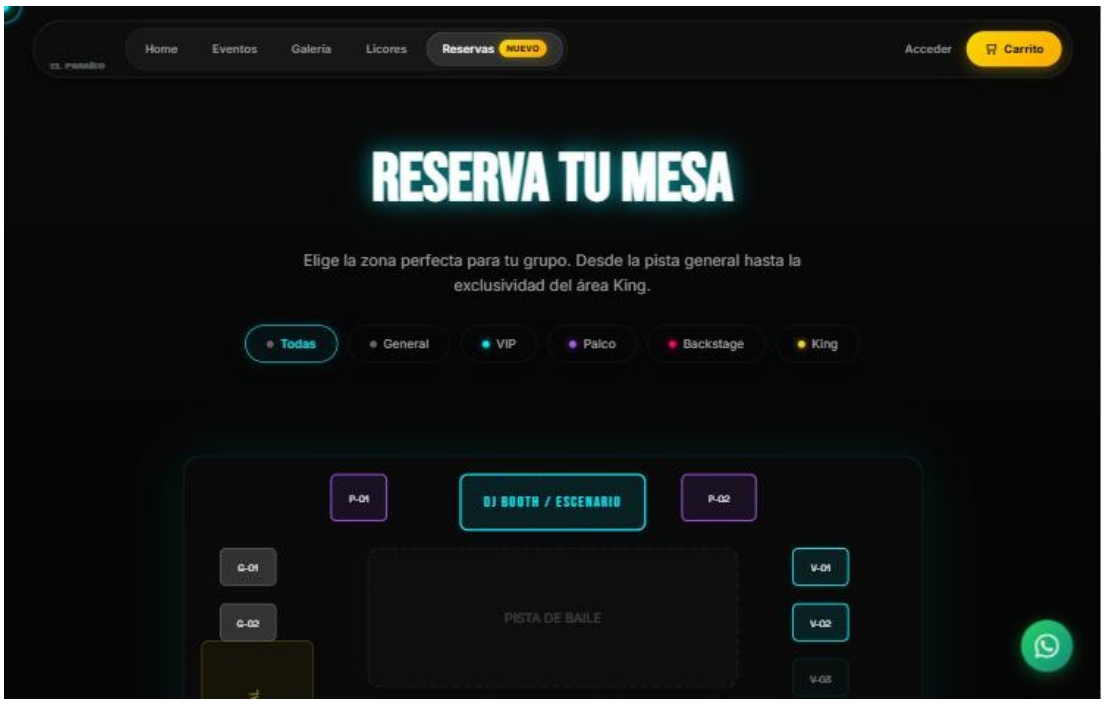


Figura 4. Mapa para reservar mesa

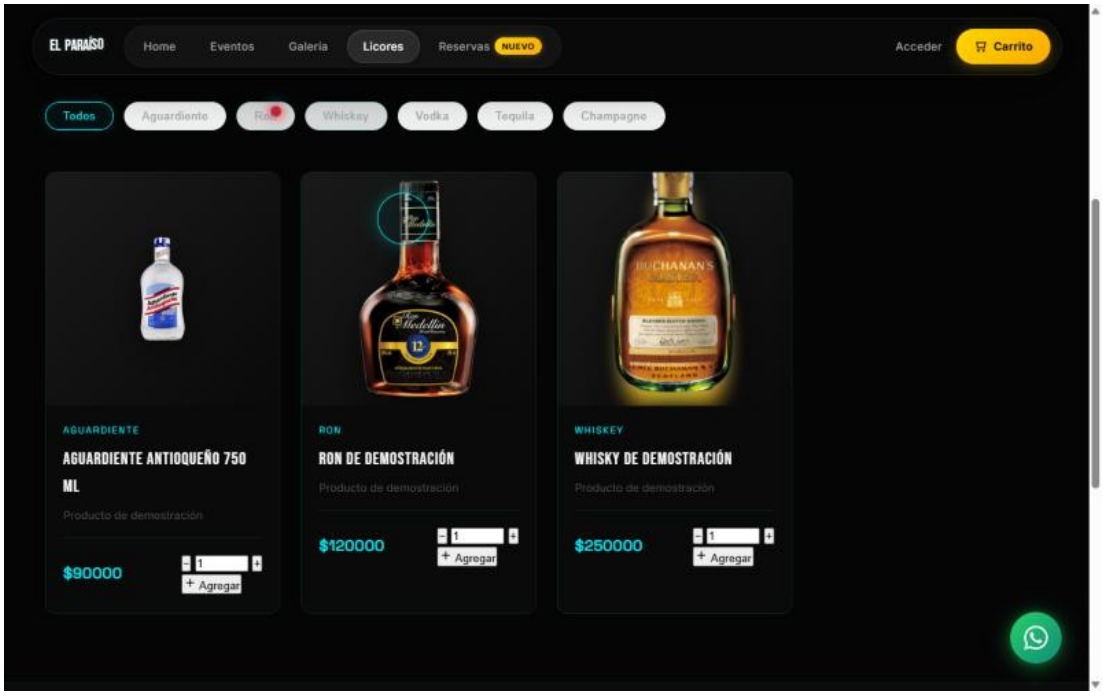


Figura 5. Carta de licores

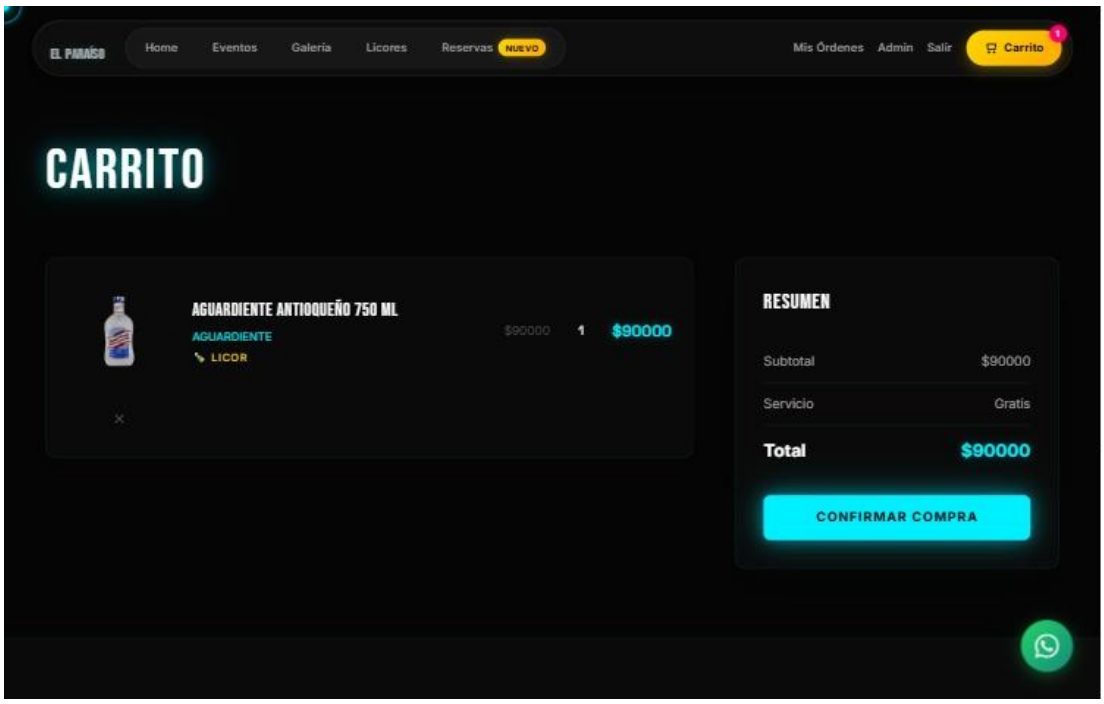


Figura 6. Carrito de compras

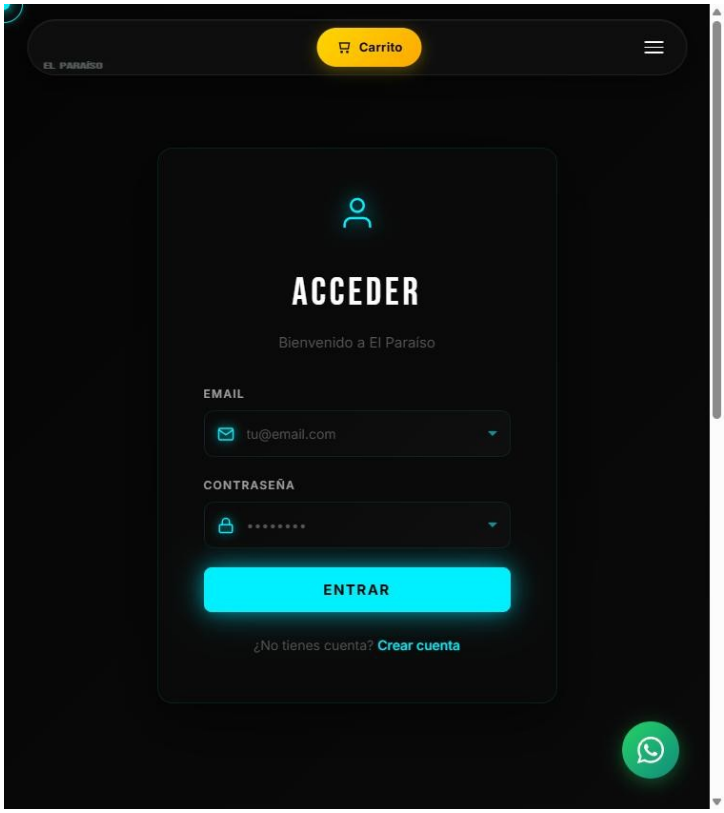


Figura 7. Inicio de sesión

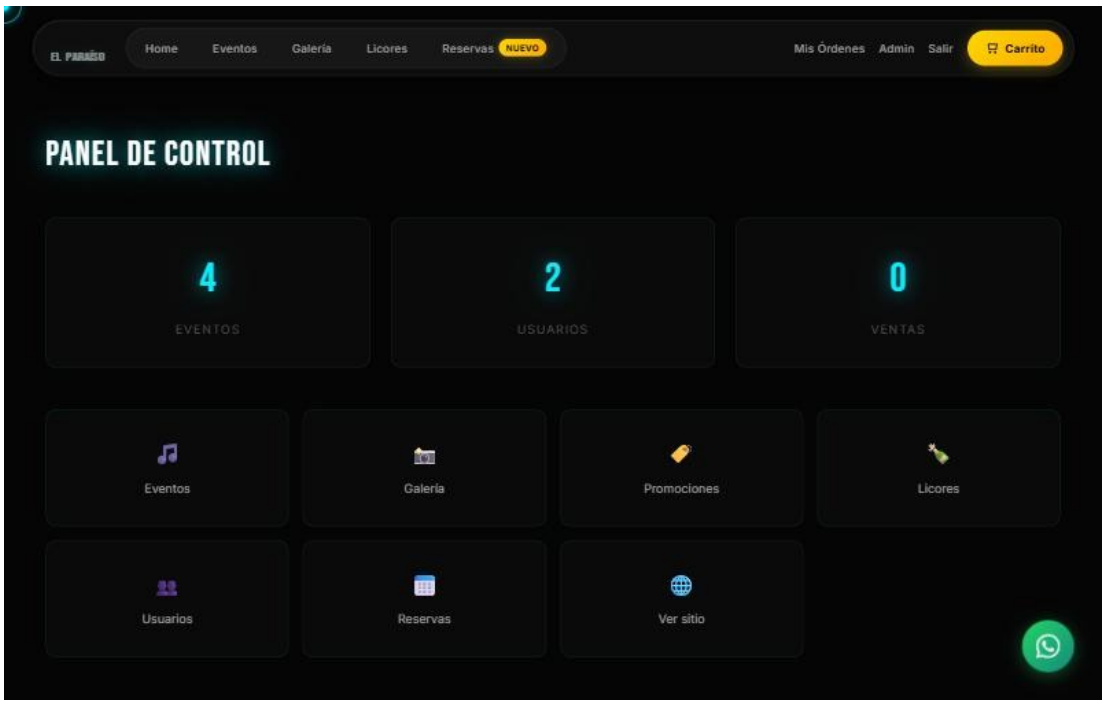


Figura 8. Panel de administración

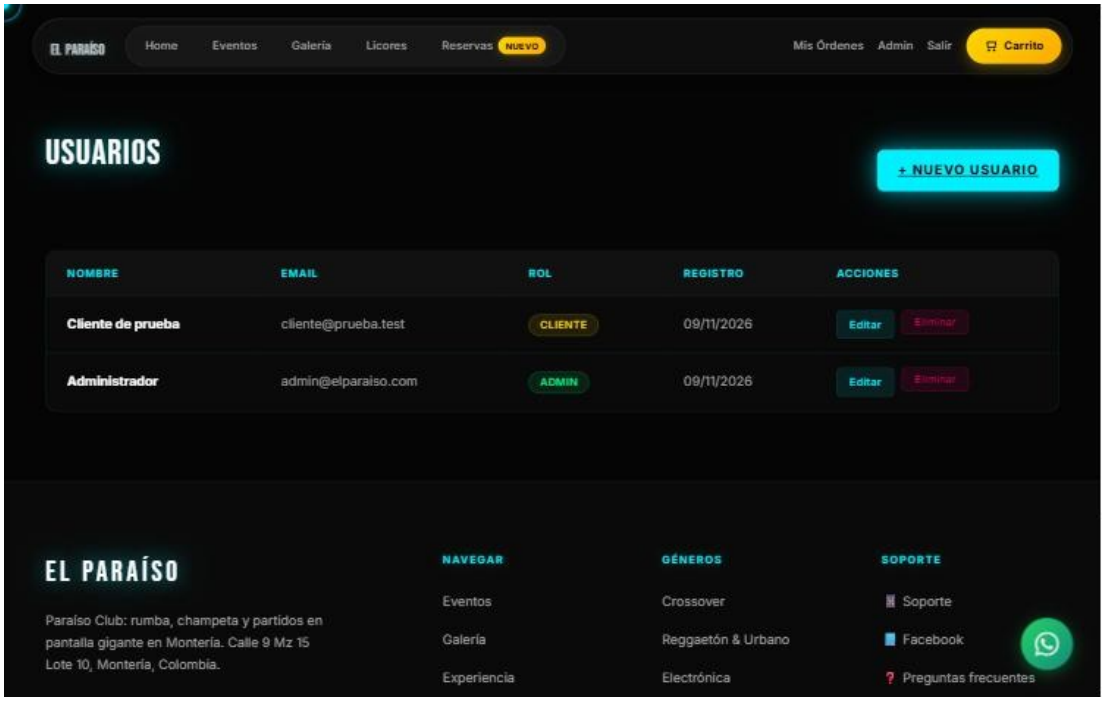


Figura 9. Administración de usuarios

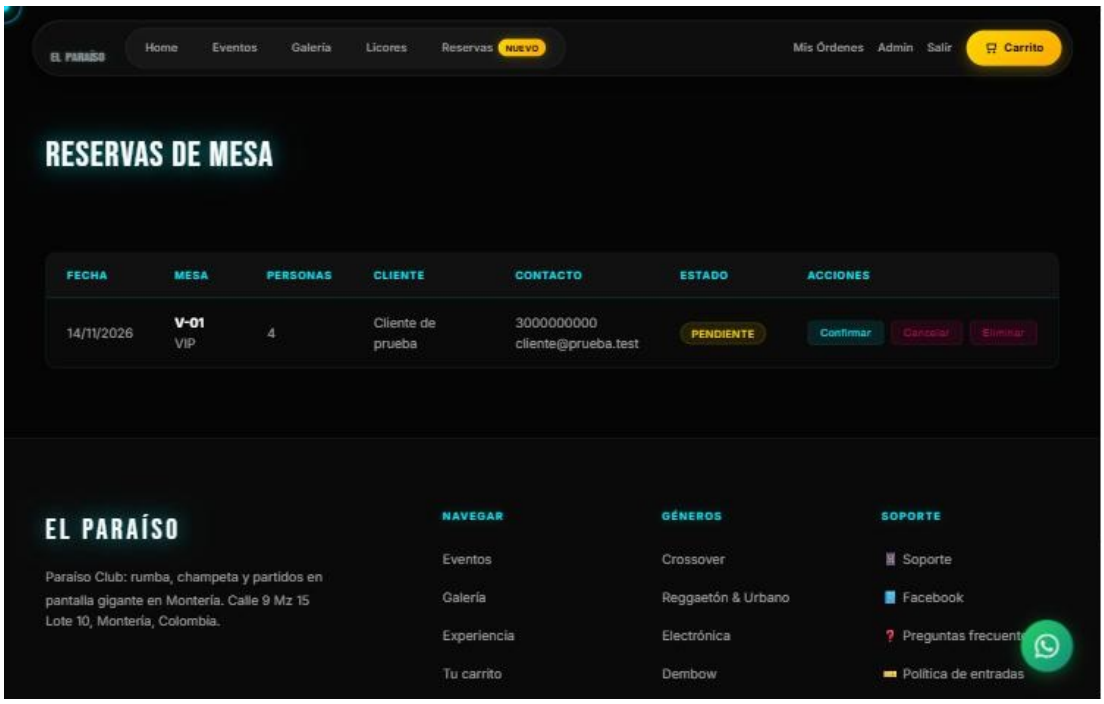


Figura 10. Administración de reservas

Teoría del color

La paleta parte del ambiente nocturno de una discoteca. El logo de Paraíso Club usa amarillo, blanco y negro; por eso el fondo es negro y el botón principal del carrito es dorado. Los acentos neón evocan las luces de la pista.

Color	Hex	Nombre	Uso y significado
	#050505	Fondo principal	Elegancia y ambiente nocturno; hace resaltar los acentos
	#00F0FF	Cian neón	Luces de pista; títulos, enlaces y acciones
	#FFD700	Dorado	Se relaciona con el amarillo del logo; carrito y categoría King
	#FF006E	Magenta	Energía y alerta; acciones de eliminar y errores
	#00FF88	Verde	Confirmación; estados correctos
	#FFFFFF	Blanco	Máximo contraste para los textos principales

Se aplica un esquema de **contraste alto**: sobre fondo casi negro, el texto blanco y los acentos saturados dirigen la atención hacia las acciones importantes. Los colores cálidos (dorado, magenta) y fríos (cian) se usan de forma separada para distinguir compra, alerta y navegación.

Tipografía

Fuente	Uso
Bebas Neue	Títulos; recuerda a los carteles de eventos
Inter	Textos y formularios; muy legible en pantalla
Space Grotesk	Etiquetas y precios

10. Herramientas utilizadas

Herramienta	Uso
Python y Flask	Servidor y rutas
Flask-SQLAlchemy	Acceso a la base de datos
Flask-Login	Autenticación y sesiones
Flask-WTF	Protección CSRF
Flask-Limiter	Límite de intentos de acceso
Werkzeug	Cifrado de contraseñas
SQLite, MariaDB y PostgreSQL	Desarrollo, verificación del script SQL y producción
HTML, CSS, JavaScript y Jinja2	Interfaz
GSAP y Lenis	Animaciones y desplazamiento suave
Git y GitHub	Control de versiones
Render y Gunicorn	Publicación en línea
Visual Studio Code	Editor de código
Claude Code (asistente de IA)	Apoyo en correcciones, pruebas y documentación

11. Seguridad

Riesgo	Medida implementada
Robo de contraseñas	Se guardan solo como hash con Werkzeug
Falsificación de peticiones (CSRF)	Token en todos los formularios y llamadas internas
Fuerza bruta en el login	Máximo 5 intentos por minuto por cuenta
Acceso no autorizado	Rutas protegidas por sesión y por rol de administrador
Datos inválidos	Validación en el servidor de nombre, correo, contraseña y cantidades
Alteración de precios	El precio de la mesa lo define el servidor
Redirección a sitios externos	Solo se aceptan destinos internos después del login
Archivos maliciosos	Lista de extensiones permitidas y nombres saneados
Sesión abierta	Cierre por POST y páginas privadas sin caché

12. Pruebas

Se ejecutó un conjunto de **84 verificaciones automatizadas** que recorren la aplicación como lo haría un usuario, con la protección CSRF activa. Todas fueron correctas tanto con SQLite como con las tablas creadas por elparaiso.sql en MariaDB. También se revisó el funcionamiento en un navegador real.

Área	Casos probados	Resultado
Páginas públicas	Carga de páginas y datos de la empresa	Correcto
CSRF	Peticiones sin token rechazadas	Correcto
Registro	Campos vacíos, correo inválido y contraseña corta	Correcto
Login	Acceso, destino interno, destino externo bloqueado y límite de intentos	Correcto
Usuarios	Crear, editar, correo repetido, eliminar y protecciones de la propia cuenta	Correcto
Carrito y licores	Agregar, quitar, cantidades negativas y paso del carrito anónimo al login	Correcto
Reservas	Precio alterado, mesa inexistente, disponibilidad, confirmar y cancelar	Correcto
Promociones y galería	Crear, editar y código repetido	Correcto
Compra	Checkout de entradas y licores e historial de órdenes	Correcto
Cierre de sesión	GET rechazado, POST correcto y acceso posterior denegado	Correcto
Script SQL	Carga doble en MariaDB 10.4 sin errores ni duplicados	Correcto

13. Despliegue

La aplicación se publica desde GitHub en un servicio web de Render, con ejecución mediante Gunicorn. Cada cambio en la rama principal genera un nuevo despliegue. Para conservar los datos se creó una base PostgreSQL en Render, cuya versión gratuita caduca el 9 de diciembre de 2026.

Para completar: confirma aquí que la base ya quedó conectada y que el sitio final funciona. Agrega una captura del panel de Render.

14. Limitaciones y conclusiones

- El pago es simulado: registra la orden pero no conecta una pasarela real.
- Fechas y precios de los eventos cargados son de ejemplo y se ajustan desde el panel.
- El servicio gratuito de Render se suspende sin visitas y tarda en despertar.
- Las imágenes subidas al servidor gratuito pueden perderse al reiniciarse; conviene usar URLs.

La aplicación centraliza la programación, las ventas, las reservas y los usuarios de El Paraíso en un solo sistema, con acceso controlado y operaciones completas de registro, consulta, modificación y eliminación, lo que responde a la necesidad identificada en el análisis.